SCHOOL OF NATURAL AND APPLIED SCIENCES

DEPARTMENT OF COMPUTING

TO : MR. S. YUTE

FROM : MOSES SAMUEL & CHISOMO KAWINJO

REGISTRATION NO : BSC-COM-NE-33-23 & BSC-COM-NE-23-33

COURSE NAME : NETWORK PROGRAMMING AND APPLICATION DEVELOPMENT

COURSE CODE : NET322

ASSIGNMENT TITLE : IoT TELEMETRY PIPELINE & DASHBOARD

DUE DATE : 8TH APRIL, 2026

Introduction

The proposed system is an Internet of Things (IoT) telemetry platform designed for greenhouse monitoring across multiple farms. Simulated sensors periodically generate environmental readings including temperature, humidity, soil moisture, and light intensity, then transmit them to a centralized telemetry server. The server stores readings, exposes historical data through a REST API, and distributes live telemetry updates through WebSockets. The architecture combines YAML configuration files, Protocol Buffers serialization, AsyncIO concurrency, HTTP-based web services, and multiple response formats to satisfy efficiency, interoperability, and scalability requirements.

A. Serialization Choices

Different serialization technologies are used in different parts of the system because each communication layer has different operational requirements.

YAML is used for sensor configuration because it is highly human-readable and easy to maintain. Configuration files are commonly edited manually by administrators, making readability more important than compactness. YAML supports hierarchical structures naturally, allowing sensor identifiers, reporting intervals, ports, and server addresses to be represented clearly with minimal syntactic noise. Compared to XML, YAML is significantly less verbose, while remaining easier to understand than JSON for configuration-oriented tasks.

Protocol Buffers (Protobuf) are used for communication between sensors and the telemetry server because IoT telemetry systems prioritize efficiency and low bandwidth usage. Protobuf serializes structured data into a compact binary format, producing smaller payloads than JSON or XML. This improves transmission speed and reduces network overhead when many sensors publish readings concurrently. Protobuf also provides strict schema support through .proto definitions, ensuring consistent message structures across clients and servers while minimizing serialization errors.

The public REST API supports JSON and XML because external clients may have different compatibility requirements. JSON is lightweight, widely supported in web and mobile applications, and easy to parse in JavaScript environments. XML is more verbose but remains

important for interoperability with legacy enterprise systems. Supporting multiple formats through HTTP content negotiation allows the same API endpoints to serve modern and older systems without duplication of business logic.

B. Web Services Protocol

REST is the most appropriate architectural style for the historical telemetry API because the system is fundamentally resource-oriented. Resources such as sensors and readings are naturally represented using URIs including /sensors and /sensors/{id}/readings. REST also maps cleanly onto standard HTTP methods such as GET, POST, and DELETE, improving clarity and maintainability.

Another major advantage of REST is statelessness. Each request contains all information necessary for processing, reducing server-side complexity and improving scalability. Stateless communication simplifies load balancing and allows the server to handle many concurrent clients efficiently.

SOAP was not selected because it introduces unnecessary complexity for a lightweight telemetry platform. SOAP relies heavily on XML envelopes and strict messaging standards, increasing verbosity and processing overhead. While SOAP provides strong enterprise-level features, these benefits are unnecessary for simple sensor telemetry and historical queries.

RPC systems focus on invoking remote procedures or functions, such as getSensorData(), rather than manipulating resources. Although RPC may provide efficient communication, REST offers better interoperability, simpler debugging, and a more intuitive design for web-based services.

For these reasons, REST provides the best balance of simplicity, scalability, interoperability, and maintainability for the telemetry platform.

C. AsyncIO vs Threading

The telemetry platform is primarily an I/O-bound application because most operations involve waiting for external events rather than performing CPU-intensive computations. The server continuously waits for sensor transmissions, REST requests, database access, and

WebSocket traffic. In such scenarios, AsyncIO provides more efficient concurrency than thread-based designs.

AsyncIO uses coroutines coordinated by an event loop. When a coroutine encounters an `await` statement, execution is suspended temporarily while the event loop processes other pending tasks. This cooperative multitasking model enables a single thread to manage many simultaneous network connections efficiently.

Using a thread-per-connection model would increase memory consumption and context-switching overhead significantly, especially if hundreds or thousands of sensors connected simultaneously. Threads also introduce synchronization complexity and race-condition risks when accessing shared resources.

AsyncIO is therefore better suited for scalable network servers with many concurrent clients. However, threads or multiprocessing would be preferable for CPU-bound workloads such as image processing, encryption, scientific computation, or machine-learning inference, where parallel execution across multiple CPU cores becomes beneficial.

If blocking calls were introduced into the AsyncIO event loop, overall responsiveness would degrade because the event loop would be unable to schedule other coroutines while waiting for the blocking operation to complete.

D. HTTP Mechanics

One important HTTP mechanism in this system is content negotiation. Different clients require telemetry data in different formats including JSON, XML, and YAML. Clients specify their preferred representation using the HTTP `Accept` header, such as `Accept: application/json` or `Accept: application/xml`. The server examines this header and serializes the response accordingly. This mechanism allows one REST API implementation to support multiple client ecosystems while preserving consistent endpoint behavior.

Another important mechanism is the WebSocket upgrade handshake. WebSocket communication begins as a standard HTTP request containing upgrade headers such as `Upgrade: websocket` and `Connection: Upgrade`. Once the server accepts the upgrade request, the protocol transitions from stateless HTTP request-response communication into a

persistent full-duplex connection. This persistent connection allows the telemetry server to push live sensor readings to dashboard clients immediately without repeated polling requests.

Cookies are also used within the REST API to identify client sessions. When a client first communicates with the server, the server may generate a session identifier and return it using the Set-Cookie header. The client automatically includes the cookie in future requests, allowing the server to associate requests with a specific session. This can be useful for tracking subscriptions, preferences, authentication state, or request history.

Persistent HTTP connections further improve efficiency because multiple requests can reuse the same TCP connection instead of repeatedly establishing and closing new connections, thereby reducing latency and network overhead.

Conclusion

The greenhouse telemetry platform integrates multiple networking and serialization technologies to satisfy different operational requirements. YAML improves configuration readability and maintainability, while Protobuf provides efficient binary serialization for sensor communication. RESTful web services combined with JSON and XML maximize interoperability for external clients. AsyncIO enables scalable concurrent communication for large numbers of sensors and clients, while WebSockets provide low-latency real-time telemetry delivery. Together, these technologies create a flexible, efficient, and scalable IoT monitoring architecture

References

1. Google Protocol Buffers Documentation
2. Python AsyncIO Documentation

   MDN Web Docs — HTTP Content Negotiation

3. MDN Web Docs — WebSockets API
4. YAML Official Specification

5. Roy Fielding — Architectural Styles and the Design of Network-based Software Architectures